BAIL security

MidasSwap

Update Audit (differential)

FINAL REPORT

November '2025

Disclaimer:

Security assessment projects are time-boxed and often reliant on information that may be provided by a client, its affiliates, or its partners. As a result, the findings documented in this report should not be considered a comprehensive list of security issues, flaws, or defects in the target system or codebase.

The content of this assessment is not an investment. The information provided in this report is for general informational purposes only and is not intended as investment, legal, financial, regulatory, or tax advice. The report is based on a limited review of the materials and documentation provided at the time of the audit, and the audit results may not be complete or identify all possible vulnerabilities or issues. The audit is provided on an "as-is," "where-is," and "as-available" basis, and the use of blockchain technology is subject to unknown risks and flaws.

The audit does not constitute an endorsement of any particular project or team, and we make no warranties, expressed or implied, regarding the accuracy, reliability, completeness, or availability of the report, its content, or any associated services or products. We disclaim all warranties, including the implied warranties of merchantability, fitness for a particular purpose, and non-infringement.

We assume no responsibility for any product or service advertised or offered by a third party through the report, any open-source or third-party software, code, libraries, materials, or information linked to, called by, referenced by, or accessible through the report, its content, and the related services and products. We will not be liable for any loss or damages incurred as a result of the use or reliance on the audit report or the smart contract.

The contract owner is responsible for making their own decisions based on the audit report and should seek additional professional advice if needed. The audit firm or individual assumes no liability for any loss or damages incurred as a result of the use or reliance on the audit report or the smart contract. The contract owner agrees to indemnify and hold harmless the audit firm or individual from any and all claims, damages, expenses, or liabilities arising from the use or reliance on the audit report or the smart contract.

By engaging in a smart contract audit, the contract owner acknowledges and agrees to the terms of this disclaimer.

1. Project Details

Important:
Please ensure that the deployed contract matches the source-code of the last commit hash.

Project	MidasSwap - Update Audit (differential)
Website	www.midasswap.xyz
Language	Solidity
Methods	Manual Analysis
Github repository	https://github.com/jk889971/MidasContracts/tree/ace098f0e42ffdfe3b69eb59bb055437249aae06 core/MidasFactory.sol (differential) core/UniswapV2Pair.sol (differential) periphery/MidasRouter.sol (differential) periphery/ReferralRegistry.sol (full)
Resolution 1	

2. Detection Overview

Severity	Found	Resolved	Partially Resolved	Acknowledged (no change made)	Failed resolution	Open
High	2	2				
Medium						
Low						
Informational	11	3		8		
Governance						
Total	13	5		8		

2.1 Detection Definitions

Severity	Description
High	The problem poses a significant threat to the confidentiality of a considerable number of users' sensitive data. It also has the potential to cause severe damage to the client's reputation or result in substantial financial losses for both the client and the affected users.
Medium	While medium level vulnerabilities may not be easy to exploit, they can still have a major impact on the execution of a smart contract. For instance, they may allow public access to critical functions, which could lead to serious consequences.
Low	Poses a very low-level risk to the project or users. Nevertheless the issue should be fixed immediately
Informational	Effects are small and do not post an immediate danger to the project or users
Governance	Governance privileges which can directly result in a loss of funds or other potential undesired behavior

3. Detection

Bailsec was tasked with a differential audit of Uniswap peripheral/core contracts:

Original code:

https://github.com/Uniswap/v2-periphery/tree/dda62473e2da448bc9cb8f4514dadda4aeede5f4/contracts (UniswapV2Router02)

https://github.com/Uniswap/v2-core/tree/1136544ac842ff48ae0b1b939701436598d74075/contracts (UniswapV2Factory, UniswapV2Pair)

New Commit:

https://github.com/jk889971/MidasContracts/tree/ace098f0e42ffdfe3b69eb59bb055437249aae06 (including all contracts in scope)

Files in Scope:

1. core/MidasFactory.sol - https://www.diffchecker.com/KGaYrbro/
2. core/UniswapV2Pair.sol - https://www.diffchecker.com/ygtJXPmh/
3. periphery/MidasRouter.sol - https://diffchecker.com/ygtJXPmh
4. periphery/ReferralRegistry.sol - full audit

Primary Update Overview:

The key update in this differential audit involves the implementation of a referral fee for swaps within the MidasRouter (and the corresponding ReferralRegistry) as well as a modification of the original UniswapV2Pair to take a 0.2% fee instead of 0.3%

Mainly:

a) UniswapV2Pair
b) MidasRouter

Security Considerations: The main security concern comes from the custom referral implementation within the MidasRouter.

MidasFactory

The MidasFactory is forked from the original UniswapFactory and the only change is variable renaming: https://www.diffchecker.com/KGaYrbro/

No issues found.

UniswapV2Pair

The UniswapV2Pair is forked from the original UniswapV2Pair and the only change is the fee percentage reduction from 0.3% to 0.2%: https://www.diffchecker.com/ygtJXPmh/

No issues found.

MidasRouter

The MidasRouter is a custom UniswapV2Router contract which enforces an extra fee of 0.1% on each swap. The general rule of thumb is that this fee is always taken as the input token, unless a swap with ETH out is executed, which then takes the fee as WETH.

All known swap functions have been modified to support the referral fee, the logic is as follows:

Exact Input: The fee is deducted from the input amount and the net amount is used for subsequent pair swaps.

Exact Output: The input amount is calculated via the standard Uniswap logic and the fee is calculated and applied on top of the required input amount.

For ETH swaps, the logic deviates from the above mentioned mechanism.

Below, we will explain the exact fee mechanics of each swap function in specifics:

a) swapExactTokensForTokensWithReferrer
   i) Fee is calculated based on amountIn
   ii) Fee is distributed
   iii) Leftover amount is transferred to pair

b) swapTokensForExactTokensWithReferrer
   i) Calculate required input amount
   ii) Calculate fee on top of required input amount
   iii) Re-calculate fee from amountInWithFee
   iv) Fee is distributed
   v) Leftover amount is transferred to pair

c) swapExactETHForTokensWithReferrer
   i) Fee is calculated based on amountIn
   ii) Fee is distributed
   iii) Leftover amount is transferred to pair

d) swapTokensForExactETHWithReferrer
   i) Calculate required output amount with fee on top
   ii) Calculate required input amount

iii) Execute swap
iv) Transfer out amount - fee to recipient
v) Fee is distributed

e) swapExactTokensForETHWithReferrer
i) Calculate amountOut
ii) Calculate fee from amountOut
iii) Execute swap
iv) Distribute fee
v) Transfer out leftover

f) swapETHForExactTokensWithReferrer
i) Calculate amountIn
ii) Calculate amountInWithFee
iii) Fee is distributed
iv) Swap is executed

g) swapExactTokensForTokensSupportingFeeOnTransferTokensWithReferrer
i) Fee is calculated based on amountIn
ii) Fee is distributed
iii) Leftover amount is transferred to pair

h) swapExactETHForTokensSupportingFeeOnTransferTokensWithReferrer
i) Fee is calculated based on amountIn
ii) Fee is distributed
iii) Leftover amount is transferred to pair

i) swapExactTokensForETHSupportingFeeOnTransferTokensWithReferrer
i) Swap is executed
ii) Output amount is determined
iii) Fee is calculated based on amountOut
iv) Fee is distributed
v) Leftover is transferred to user

Core Invariants:

INV 1: Fee must be always taken in WETH (for ETH swaps) or in the input token

INV 2: amountIn parameter within _takeInputAndSplitFeeThenSendToFirstPair must always be the amount including the fee

INV 3: _distributePlatformFee must allocate any remaining dust to devAmt

INV 4: _takeInputAndSplitFeeThenSendToFirstPair must always be called with token at path[0]

INV 5: _takeInputAndSplitFeeThenSendToFirstPair must always be called with msg.sender as trader parameter

Privileged Functions

- setDevWallet
- setFallbackWallet

Issue_01	Incorrect implementation of swapExactTokensForTokensSupportingFeeOnTransferTokenWithReferrer
Severity	**High**
Description	The swapExactTokensForTokensSupportingFeeOnTransferTokenWithReferrer is supposed to be compatible with transfer-tax tokens but transfers the token first to the router and then calculates the fee and leftover amount based on amountIn. This is incorrect as the actual received amount will be less than amountIn and thus the execution reverts. Please note that furthermore, due to the double transfer, the tax in the input token is taken twice (if the root-cause above is fixed). This is however a design consideration and can be accepted.
Recommendations	Consider caching the balance before and after (or even only after) to determine the received balance of the first token.
Comments / Resolution	Fixed by transferring input tokens to the pair directly, thus avoiding double tax. Additionally, the platform fee is now charged on the output token instead.

Issue_02	Insufficiently thought out design for token burns
Severity	**High**
Description	Each swap burns a part of the 0.1% fee. This implementation indeed makes sense for protocol tokens (such as MIDAS). But it will result in loss of funds if the input token is no MIDAS token. The same is true for ETH swaps, as essentially WETH gets burned.
Recommendations	Consider simply switching the burn address with a protocol-owned address. Governance can then manually decide which tokens to keep and which tokens to burn.
Comments / Resolution	Fixed by using a configurable burn wallet address instead of a dead address.

Issue_03	Bypass of input token fee by routing through pool with custom token
Severity	**Informational**
Description	Currently, the fee is always taken on the input token - unless an ETH swap is executed. This can be trivially bypassed by routing through a pool with a custom token where the custom token is received as fee which can then be simply transferred out from the recipients back to the swapper/pool/wherever.
Recommendations	Since the referral logic is introduced via the frontend and relies on such while at the same time direct UniswapV2Pair.swap calls are also possible (and should remain possible for various reasons), we simply recommend acknowledging this and do not consider it a real issue.
Comments / Resolution	Acknowledged.

Issue_04	Bypass of WETH fee payment via swapExactTokensForTokens function with WETH as output
Severity	**Informational**
Description	An important invariant is that any swap with ETH should take the fee always in WETH (and not in the input token only). This can be trivially bypassed via swapExactTokensForTokens by using WETH as output token while at the same time, the fee is taken in the input token
Recommendations	Consider acknowledging this issue.
Comments / Resolution	Acknowledged.

Issue_05	Sybil attack vulnerability
Severity	**Informational**
Description	As with each referral system, it is possible to execute a sybil-attack by setting another owned address as referrer. This is a system limitation and cannot be resolved.
Recommendations	Consider acknowledging this issue
Comments / Resolution	Acknowledged.

Issue_06	Redundant WETH balance cache during swapExactTokensForETHSupportingFeeOnTransferTokensWithReferrer
Severity	**Informational**
Description	Currently, the aforementioned function executes a pre/post balance cache around the swap execution. However, since the swap proceeds will anyways be transferred to the router, it is simpler to use the final WETH balance. See Uniswap implementation
Recommendations	Consider following the Uniswap implementation
Comments / Resolution	Acknowledged.

Issue_07	Reentrancy concerns for ERC777 tokens
Severity	**Informational**
Description	Currently, there are functions which are transferring the fee token to all different recipients, which allows for reentering during the control-flow. While we were not able to identify an issue from this - specifically because the router does not store any state - we still recommend the development team to look into potential paths. The following functions are affected: - swapExactTokensForTokensWithReferrer - swapTokensForExactTokensWithReferrer - swapExactTokensForTokensSupportingFeeOnTransferTokensWithReferrer
Recommendations	Consider further inspecting potential exploit paths due to this root-cause.
Comments / Resolution	Acknowledged.

Issue_08	DoS of swaps in case of blacklisted referrer/devWallet
Severity	**Informational**
Description	It is theoretically possible that swaps revert if one of both addresses is blacklisted for the fee token. Specifically an issue is existing for the referrer address as this cannot be changed like the devWallet.
Recommendations	We recommend acknowledging this issue, it is very unlikely to happen in reality and changes will alter the control-flow and introduce risk of other issues.
Comments / Resolution	Acknowledged.

Issue_09	Redundant duplicate token transfers within swapTokensForExactETHWithReferrer/swapExactTokensForETHWithReferrer
Severity	**Informational**
Description	Both functions are executing duplicate transfers instead of simply executing one transferFrom to move funds from the caller to the first pair.
Recommendations	Consider transferring funds directly to the pair instead of to the router first.
Comments / Resolution	Fixed by following recommendation.

Issue_10	Direct pair interaction allows to bypass referral fee
Severity	**Informational**
Description	Anyone can simply call UniswapV2Pair.swap and execute swaps without paying a referral fee. This is a limitation of the design and can only be fixed by modifying the UniswapV2Pair directly.
Recommendations	Consider either modifying the swap function or acknowledging this issue. Our recommendation is to acknowledge this issue as guardian the swap function on the pair may come with unexpected side-effects.
Comments / Resolution	Acknowledged.

Issue_11	Permit griefing possibilities
Severity	**Informational**
Description	Every function which relies on permit logic can be griefed whereas anyone can simply frontrun the permit call which then results in a revert of the function. This is however a non-issue as one can simply re-execute the standard logic then because the approval has been executed.
Recommendations	Consider acknowledging this issue.
Comments / Resolution	Acknowledged.

Issue_12	Taking a fee on the input token will slightly decrease the LP fee
Severity	**Informational**
Description	Currently, the input token is taken as fee token unless it is an ETH swap. This will result in slightly decreasing the amount which is transferred to the pair and then subsequently results in less fees being paid to the pair. We consider this as non-issue as it at the same time decreases the slippage which means the price impact is lower. Furthermore, this is a solid design choice.
Recommendations	Consider acknowledging this issue.
Comments / Resolution	Fix has been implemented only on the function swapExactTokensForTokensSupportingFeeOnTransferTokenWithReferrer, which takes platform fee on the output token. Some of the remaining swap functions e.g. swapExactTokensForTokensWithReferrer continue taking fees on the input token.

ReferralRegistry

The ReferralRegistry contract is a simple registry which keeps track of the following information:

- hasTraded[user]: Whether a user has executed at least one successful trade
- referralsCount: How much addresses a user has referred
- referrerOf[user]: The referrer (if any) of an address

Core Invariants:

INV 1: referrerOf[user] is only set if referrer is not address(0), referrer is not self, referrerOf[user] has not been set and if its the first trade of a user

INV 2: hasTraded[user] must always be set on the first trade

Privileged Functions

- setRouter

Issue_13	Owner can be made immutable
Severity	**Informational**
Description	Variables which are set once in the constructor can be marked immutable to save gas.
Recommendations	Consider marking the owner as immutable.
Comments / Resolution	Resolved.

Disclaimer: This audit involves only the changes provided by the corresponding diffchecker files. Please be advised that for issues which are reported outside of the diffchecker scope, an additional resolution must be scheduled. A differential audit is always a constrained task because not the full codebase is re-audited. This will have inherent consequences if intrusive changes have side-effects on parts of a codebase/module, which is not part of the audit scope.